

Lab 01 – Introduction, Data Types, Conversions

Title: Introduction to C# Data Types and Conversions

Objectives

The primary goal of this laboratory session was to get comfortable with the Visual Studio development environment and understand how C# handles different data types and conversions. I needed to create a simple console application that could perform salary calculations while handling user input safely. This was my first step into understanding how real-world applications process data entered by users.

Implementation

I created a new console application project called SalariiConsole. The program asks the user to enter an employee's name, their hourly rate, and how many hours they worked. The name is stored as a string, while the hourly rate and hours are stored as double and int respectively.

```
using System;
```

```
namespace SalariiConsole
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Console.WriteLine("=== Sistem Calcul Salarii ===");
```

```
            Console.Write("Introduceți numele angajatului: ");
```

```
            string nume = Console.ReadLine();
```

```
            double tarifOra = 0;
```

```
            int oreLucrate = 0;
```

```
bool inputValid = false;
```

```
// Input validation using TryParse with a while loop
```

```
while (!inputValid)
```

```
{
```

```
    Console.Write("Introduceți tariful pe oră (RON): ");
```

```
    string input = Console.ReadLine();
```

```
    if (double.TryParse(input, out tarifOra) && tarifOra >= 0)
```

```
    {
```

```
        inputValid = true;
```

```
    }
```

```
    else
```

```
    {
```

```
        Console.WriteLine("Vă rugăm să introduceți un număr valid pentru tarif.");
```

```
    }
```

```
}
```

```
inputValid = false;
```

```
while (!inputValid)
```

```
{
```

```
    Console.Write("Introduceți numărul de ore lucrate: ");
```

```
    string input = Console.ReadLine();
```

```
    if (int.TryParse(input, out oreLucrate) && oreLucrate >= 0)
```

```
    {
```

```
        inputValid = true;
```

```
    }
```

```
    else
```

```
{  
    Console.WriteLine("Vă rugăm să introduceți un număr întreg valid pentru ore.");  
}  
}  
  
// Salary calculations with double precision  
  
double salariuBrut = tarifOra * oreLucrate;  
  
double procentTaxa = 0.10;  
  
double procentContributieSociala = 0.05;  
  
double taxa = salariuBrut * procentTaxa;  
  
double contributieSociala = salariuBrut * procentContributieSociala;  
  
double salariuNet = salariuBrut - taxa - contributieSociala;  
  
// Formatted output using :F2 for two decimal places  
  
Console.WriteLine("\n=== Detalii Salariu ===");  
  
Console.WriteLine($"Angajat: {nume}");  
  
Console.WriteLine($"Salariu brut: {salariuBrut:F2} RON");  
  
Console.WriteLine($"Taxă (10%): {taxa:F2} RON");  
  
Console.WriteLine($"Contribuție socială (5%): {contributieSociala:F2} RON");  
  
Console.WriteLine($"Salariu net: {salariuNet:F2} RON");  
}  
}  
}
```

Why I Used This Approach

TryParse Method: I used TryParse instead of Parse because it handles conversion errors gracefully. If the user enters something that isn't a number, the program would crash with a

`FormatException` if I used `Parse`. `TryParse` returns a boolean indicating success or failure, allowing me to handle the error without crashing.

While Loop for Validation: Wrapping the input in a while loop keeps asking for valid input until it's provided. This creates a robust user experience where the program doesn't just fail but guides the user to correct their input.

Double Precision: I used double-precision floating-point numbers for financial calculations. While `decimal` would be more precise for currency, `double` is sufficient for this learning exercise.

String Formatting with `:F2`: The `:F2` format specifier formats numbers to two decimal places, making the output look professional and easy to read.

Testing & Results

I tested the application with various inputs:

- When I entered "abc" for the hourly rate, the program prompted me again without crashing.
- When I entered valid numbers like 25.50 for rate and 40 for hours, it correctly calculated:
 - Gross: 1020.00
 - Tax (10%): 102.00
 - Social contribution (5%): 51.00
 - Net: 867.00
- Edge cases: entering 0 for hours produced a salary of 0, and negative numbers were properly rejected.

Reflection

This lab was a great refresher on fundamental C# concepts. The `TryParse` pattern is something I'll use frequently throughout this project. I also realized that handling user input carefully is crucial for creating robust applications that don't crash unexpectedly. The separation of input validation from business logic is a pattern I'll continue to apply in future labs.